EJERCICIO PRACTICO

ARQUITECTO DE SOLUCIONES BP

JONATHAN LUDEÑA

MAYO 2026

Repositorio: https://github.com/jonathanludena/arq-design-may2026-AS-JL

Tabla de contenido

1. Visión General de la Solución 4
2. Stack Tecnológico Recomendados 5
3. Aplicación móvil: opciones evaluadas 6
   Opción 1: Flutter 6
   Opción 2: React Native 6
   Elección recomendada: Flutter 6
4. Autenticación y autorización 7
   Flujo recomendado 7
5. Onboarding con reconocimiento fácil y KYC (*Know Your Customer* o Conoce a tu Cliente) 8
   Flujo propuesto 8
6. Microservicios propuestos 9
   Justificación 9
7. Patrones arquitectónicos 10
8. Persistencia para clientes frecuentes 11
   Propuesta 11
   Componentes 11
   Flujo 11
   Justificación 11
9. Auditoría 12
   Requerimiento 12
   Tecnología recomendada 12
   Patrón recomendado: 12
10. Notificaciones 13
   Diseño 13
   Flujo 13
11. Seguridad 14
   Controles recomendados 14
12. Normativa y cumplimiento 15
13. Alta disponibilidad, tolerancia a fallos y DR 16
   Alta Disponibilidad 16

DR recomendado 16
14. Observabilidad y excelencia operativa 17
Componentes 17
Métricas clave 17
15. Decisiones arquitectónicas con justificación 18
ANEXOS 21
DIAGRAMA C4 – CONTEXTO 22
DIAGRAMA C4 – CONTENEDORES 23
DIAGRAMA C4 – COMPONENTES 24
DIAGRAMA DE INFRAESTRUCTURA – AZURE 25
FLUJO DE CONSULTA DE MOVIMIENTOS 25
FLUJO DE TRANSFERENCIAS 27

1. Visión General de la Solución

El Sistema tiene 2 canales digitales:

1. SPA Web Banking
2. Aplicación móvil multiplataforma

Ambos consumen APIs expuestas mediante **Azure Front Door + WAF + Azure API Management,** autenticadas con **OAUTH 2.0 / OpenID Connect** usando el producto corporativo existente de identidad.

La lógica de negocio se organiza en microservicios desplegados en **Azure Kubernetes Service**, integrados con:

- Core Bancario
- Sistema complementario de cliente
- Servicios externos de transferencia interbancarias
- Proveedores de notificaciones
- Proveedor KYC / biometría facial
- Base de auditoría inmutable
- Cache distribuida para clientes frecuentes

Justificación: Azure API Management es adecuado como Gateway porque permite publicar APIs de forma segura y a escala, aplicar políticas, enrutar, autenticar, limitar tráfico y recolectar telemetría.

2. Stack Tecnológico Recomendados

Capa	Tecnología recomendada
Front Web	React / Angular SPA
Mobile	Flutter o React Native
Gateway global	Azure Front Door Premium
Seguridad perimetral	Azure WAF en Front Door
API Gateway	Azure API Management Premium
Runtime backend	Azure Kubernetes Service
Mensajería	Azure Service Bus
Eventos	Azure Event Grid
Cache	Azure Cache for Redis Enterprise
DB transaccional digital	Azure SQL Database
Auditoría	Azure SQL Ledger o Azure Confidential Ledger
Secrets / certificados	Azure Key Vault
Observabilidad	Azure Monitor + Application Insights + Log Analytics
CI/CD	GitHub Actions o Azure DevOps
Contenedores	Azure Container Registry
Protección red	Private Link, VNet Integration, NSG, Azure Firewall
DR	Multi-región activo-pasivo o activo-activo según criticidad

Las recomendaciones se basan en diseñar una Arquitectura con los pilares de confiabilidad, seguridad, excelencia operativa, eficiencia en el rendimiento y optimización de costos.

3. Aplicación móvil: opciones evaluadas

Opción 1: Flutter

Ventajas:

1. Compila a código nativo y suele ofrecer rendimiento consistente en iOS y Android.
2. Permite construir una UI altamente controlada desde una sola base de código

Desventajas:

1. Requiere equipo con conocimiento específico con su lenguaje de programación Dart.
2. Algunas integraciones nativas bancarias pueden requerir plugins o desarrollo nativo adicional.

Opción 2: React Native

Ventajas:

1. Reutiliza conocimiento Javascript/Typescript, útil si el equipo de desarrollo ya trabaja con React.
2. Tiene ecosistema amplio para biometría local, *push notifications*, *deep links* y seguridad móvil

Desventajas:

1. Puede requerir más ajuste nativo en casos de alto rendimiento
3. La dependencia de librerías externas debe gestionarse cuidadosamente en un entorno financiero.

Elección recomendada: Flutter

Se recomienda **Flutter** para la aplicación móvil bancaria por:

1. **Consistencia visual y de experiencia:** reduce diferencias entre Android e iOS, importante en flujos sensibles como onboarding, pagos y autenticación.
2. **Rendimiento y control de UI:** útil para formularios transaccionales, validaciones, pantallas biométricas y experiencia *mobile-first*
3. **Menor fragmentación de componentes visuales** frente a integraciones híbridas.
4. **Buena integración con biometría local** mediante canales nativos cuando se requiere seguridad avanzada.

NOTA: React Native es válido si la organización prioriza reutilización de talento React/Typescript

4. Autenticación y autorización

Flujo recomendado

Para SPA y móvil se recomienda: **OpenID Connect + OAuth 2.0 Authorization Code Flow con PKCE**

No se recomienda usar **Implicit Flow.**

La razón es que OpenID Connect usa el Authorization Code Flow para obtener un código que luego se intercambia por tokens, evitando exponer tokens directamente al *user-agent*. Además, la práctica actual para aplicaciones *browser-based* recomienda Authorization Code Flow con PKCE en lugar de Implicit Flow

Flujo para SPA:

1. Usuario abre SPA
2. SPA redirige al Identity Provider corporativo
3. Login con usuario/clave + MFA adaptativo
4. Authorization Code + PKCE
5. SPA recibe tokens.
6. Tokens se almacenan preferentemente en memoria, NO en el *localStorage*.
7. Renovación mediante *refresh token* rotativo o *silent re-auth* controlado
8. Llamadas al API Gateway con *access token*
9. API Management valida *JWT*, *scopes*, *audience*, *issuer* y *claims*

Flujo para móvil:

1. App móvil inicia login con navegador del sistema
2. Authorization Code + PKCE
3. El token se guarda en Keychain/Keystore
4. Para reingreso, se habilita biometría local.
5. La biometría desbloquea credenciales locales o sesión, pero no reemplaza la autenticación remota.
6. Operaciones sensibles requiere *step-up authentication*

NOTA: OWASP MASVS recomienda que apps móviles usen protocolos seguros de autenticación/autorización y autenticación local segura según las mejores prácticas de cada plataforma.

5. Onboarding con reconocimiento fácil y KYC (*Know Your Customer* o Conoce a tu Cliente)

Flujo propuesto

1. Usuario instala app móvil
2. Selecciona “Crear cuenta digital”
3. App invoca **Onboarding Service**
4. Onboarding Service genera una sesión KYC
5. Usuario captura documento de identidad
6. Usuario realiza prueba de vida facial
7. KYC Provider valida:
   - Documento
   - Selfie
   - Liveness detection
   - Face match
   - Riesgo de fraude
   - Listas restrictivas, si aplica
8. Resultado KYC se guarda como evidencia controlada
9. Se crea perfil digital del cliente
10. Se registra usuario en el Identity Provider corporativo
11. Usuario configura:
    - Usuario y clave
    - MFA
    - Biometría local
    - Dispositivo confiable
12. Se emite credencial digital de acceso

Herramientas KYC recomendadas:

Proveedor	Uso recomendado
Jumio	Verificación de identidad, biometría, liveness y prevención de fraude
Entrust / Onfido	Validación documental + biometría facial
Trulioo	Cobertura global de identidad y fuentes de datos
Microsoft Entra Verified ID	Credenciales verificables y Face Check integrado al ecosistema Microsoft

6. Microservicios propuestos

Además de los 3 servicios iniciales, se propone ampliar el dominio para mejorar cohesión, rendimiento y escalabilidad.

Servicio	Responsabilidad
Customer Profile Service	Datos básicos del cliente
Customer Detail Aggregator	Complementa información desde el sistema independiente
Product Service	Productos financieros del cliente
Movement Service	Histórico de movimientos
Transfer Service	Transferencias propias e interbancarias
Payment Service	Pagos de servicios, tarjetas, préstamos
Notification Service	Envío de email, SMS, push, WhatsApp
Audit Service	Registro inmutable de acciones
Onboarding Service	Alta digital de clientes
KYC Adapter Service	Integración con proveedor KYC
Authorization Policy Service	Reglas de autorización transaccional
Limits & Risk Service	Límites, score de riesgo, antifraude
Frequent Customer Cache Service	Cache y vistas materializadas
Consent Service	Consentimientos y tratamiento de datos
Device Trust Service	Gestión de dispositivos confiables
BFF Web	Backend específico para SPA
BFF Mobile	Backend específico para móvil

Justificación

1. **Separación por dominio:** evita servicios genéricos demasiado grandes y facilita evolución independiente
2. **Bajo acoplamiento:** cada servicio encapsula lógica, datos y contratos
3. **Escalabilidad selectiva:** Ejemplo: Movement Service y Transfer Service pueden escalar diferente
4. **Mejor resiliencia:** Ejemplo: fallas en notificaciones o KYC no bloquean consultas de movimientos

7. Patrones arquitectónicos

Patrón	Uso
API Gateway	Entrada central, seguridad, throttling y versionamiento
BFF	Diferenciar necesidades de SPA y móvil
CQRS	Separar consultas frecuentes de comandos transaccionales
Cache-Aside	Cache de clientes frecuentes
Event-Driven Architecture	Auditoría, notificaciones y eventos transaccionales
Transactional Outbox	Garantizar publicación confiable de eventos
Saga	Orquestar transferencias y pagos distribuidos
Circuit Breaker	Proteger contra fallos de Core o terceros
Retry con backoff	Reintentos controlados
Bulkhead	Aislar recursos por dominio
Adapter	Integración con Core, KYC, notificaciones y transferencias externas
Strategy	Selección dinámica de proveedor de notificación
Repository	Acceso a datos desacoplado
Materialized View	Consultas rápidas de clientes frecuentes

NOTA:

- **Azure Service Bus** es un bróker empresarial administrado con colas y tópicos *publish-subscribe*, útil para desacoplar servicios y coordinar trabajo confiable. **Event Grid** es adecuado para distribución de eventos mediante pub/sub administrado

8. Persistencia para clientes frecuentes

Propuesta

Usar combinación de:

1. Cache-Aside Pattern
2. CQRS
3. Materialized Views
4. Event-driven cache invalidation

Componentes

Componente	Función
Redis Cache	Guarda datos frecuentes: perfil resumido, productos, últimas consultas
Customer Profile Service	Consulta cache primero
Movement Query Service	Lee vista optimizada
Event Bus	Publica cambios del cliente/productos/movimientos
Cache Invalidation Worker	Invalida o refresca datos
Materialized View Store	Modelo optimizado para lectura
Audit Service	Registra acceso a información sensible

Flujo

1. Cliente frecuente consulta movimientos
2. API Gateway enruta al BFF
3. BFF llama a Movement Query Service
4. Movement Query Service revisa Redis
5. Si existe dato válido, responde
6. Si no existe, consulta Core o vista materializada
7. Guarda en cache con TTL corto
8. Si ocurre movimiento nuevo, se publica evento
9. Cache Invalidation Worker actualiza o invalida la cache

Justificación

1. **Mejora latencia:** evita consultar Core en cada solicitud
2. **Reduce carga en sistemas legados:** protege el Code bancario
3. **Consistencia controlada:** TTL + eventos reducen datos obsoletos
4. **Escalabilidad de lecturas:** CQRS permite optimizar consultas sin afectar comandos

9. Auditoría

Requerimiento

Registrar todas las acciones del cliente:

- Login
- Consulta de movimientos
- Intentos fallidos
- Transferencias
- Pagos
- Cambios de datos
- Dispositivo usado
- IP, canal, geolocalización aproximada
- Resultado de autorización
- Consentimientos
- Eventos KYC
- Notificaciones enviadas

Tecnología recomendada

Para auditoría financiera se recomienda:

1. **Azure SQL Ledger**, si se requiere modelo relacional verificable
2. **Azure Confidential Ledger,** si se requiere almacenamiento *append-only*, inmutable y *tamper-proof*

Azure Confidential Ledger ofrece un almacén auditable con inmutabilidad, resistencia a manipulación y operaciones *append-only*. Azure SQL también soporta capacidades ledger para escenarios donde se requiere integridad e inmutabilidad verificable.

Patrón recomendado:

Event Sourcing parcial + Append-only Audit Log

No se propone Event Sourcing completo para todo el core digital, porque incrementaría complejidad. Sí se propone registrar eventos de auditoría inmutable como fuente verificable de trazabilidad.

10. Notificaciones

La norma exige notificar movimientos realizados. Se recomienda mínimo:

1. Push Notification
2. Email
3. SMS como respaldo
4. Opcional: WhatsApp Business, si la regulación y consentimiento lo permiten

Diseño

Notification Service debe implementar:

- Strategy Pattern para elegir canal
- Provider Adapter para proveedores externos
- Fallback entre proveedores
- Cola de notificaciones
- Dead-letter queue
- Reintentos con backoff
- Plantillas versionadas
- Registro de entrega
- Trazabilidad en auditoría

Flujo

1. Transfer Service confirma transferencia
2. Publica evento *TransferCompleted*
3. Notification Service consume evento
4. Evalúa preferencias y consentimientos
5. Envía push
6. Sí falla, envía SMS o email
7. Registra resultado
8. Publica *NotificationSent* o *NotificationFailed*

11. Seguridad

Controles recomendados

Área	Control
Perímetro	Azure Front Door + WAF
APIs	OAuth2/OIDC, JWT validation, scopes, rate limit
Red	Private Link, subnets privadas, NSG
Secrets	Azure Key Vault
Datos	Cifrado en tránsito y reposo
DB	TDE, Always Encrypted si aplica
Microservicios	mTLS interno o service mesh
Usuario	MFA adaptativo, biometría local, device trust
Operaciones sensibles	Step-up authentication
Auditoría	Append-only, inmutable
DevSecOps	SAST, DAST, dependency scanning
Protección móvil	Jailbreak/root detection, certificate pinning, secure storage
Sesiones	Refresh token rotation, token revocation
Fraude	Límites, score de riesgo, monitoreo transaccional

NOTA: Azure Front Door con WAF protege aplicaciones frente a vulnerabilidades comunes y ayuda a mantener alta disponibilidad y cumplimiento. Azure Key Vault permite almacenar claves, secretos y certificados

12. Normativa y cumplimiento

Como entidad financiera, se deben considerar:

1. **Protección de datos personales**
    a. Consentimiento explícito
    b. Finalidad del tratamiento
    c. Derecho de acceso, rectificación y eliminación cuando aplique
    d. Minimización de datos
    e. Retención regulada
2. **Seguridad financiera:**
    a. Autenticación fuerte
    b. MFA
    c. Gestión de sesiones
    d. Trazabilidad completa
    e. No repudio en operaciones
3. **PCI DSS**
    a. Si se procesan tarjetas o datos asociados
    b. No almacenar PAN/CVV salvo autorización y controles estrictos
4. **AML / KYC**
    a. Identificación del cliente
    b. Verificación documental
    c. Listas restrictivas
    d. Personas políticamente expuestas
    e. Monitoreo de fraude
5. **OWASP ASVS / MASVS**
    a. Web y APIs
    b. Seguridad móvil
    c. Almacenamiento seguro
    d. Autenticación local segura
6. **ISO 27001 / ISO 27002**
    a. Gobierno de seguridad
    b. Gestión de riesgos
    c. Controles de acceso
    d. Gestión de incidentes
7. **Registro y conservación**
    a. Auditoría inmutable
    b. Logs protegidos
    c. Retención según regulación local

13. Alta disponibilidad, tolerancia a fallos y DR

Alta Disponibilidad

Capa	Estrategia
Front	Azure Front Door global
WAF	Front Door Premium con reglas administradas
API Gateway	API Management Premium multi-región
Backend	AKS con Availability Zones
Nodos	Node pools separados por dominio
Pods	HPA, Pod Disruption Budgets
Mensajería	Service Bus Premium
Cache	Redis Enterprise con réplica
SQL	Geo-replicación activa
Auditoría	Ledger inmutable con replicación
Secretos	Key Vault con redundancia
Observabilidad	Monitor + Log Analytics

NOTA: AKS puede distribuir nodos entre Availability Zones para mejorar confiabilidad ante fallos zonales. Azure SQL Database soporta geo-replicación activa para replicar datos desde una base primaria hacia una secundaria legible en otra región.

DR recomendado

Modelo recomendado inicial:

Activo-pasivo caliente

- Región primaria: East US 2 o Brasil, según latencia/regulación
- Región secundaria: Central US, South Central US o región Azure compatible
- RPO objetivo: minutos
- RTO objetivo: menor a 1 hora para canales digitales críticos
- Front Door enruta a región secundaria ante desastre
- Bases con geo-replicación
- IaC con Terraform/Bicep para reconstrucción
- Backups probados periódicamente

NOTA: Para banca crítica 24/7 y presupuesto alto, se puede evolucionar a **activo-activo** para consultas y **activo-pasivo controlado** para comandos transaccionales.

14. Observabilidad y excelencia operativa

Componentes

- Azure Monitor
- Application Insights
- Log Analytics Workspace
- Distributed tracing con OpenTelemetry
- Dashboards con dominio
- Alertas por SLO
- Synthetic monitoring
- Health checks
- Runbooks automatizados
- Chaos Testing controlado
- Blue/Green deployments
- Canary releases
- Feature flags
- GitOps

Métricas clave

Métrica	Objetivo
Disponibilidad digital	99.9% o superior
Latencia consulta movimientos	p95 < 500 ms con cache
Latencia transferencias	p95 < 2 s sin incluir red externa
Error rate APIs	< 1%
Fallos de notificación	< 0.5%
Tiempo detección incidente	< 5 min
Tiempo recuperación	Según RTO definido
Eventos auditados	100% de acciones sensibles

15. Decisiones arquitectónicas con justificación

Decisión	Opciones evaluadas	Elección	Justificación teórica
Gateway	NGINX, Kong, Azure API Management	Azure API Management	1. Servicio administrado reduce operación. 2. Permite políticas centralizadas: JWT, throttling, cuotas, transformación y versionado.
Perímetro	App Gateway, Front Door, CDN simple	Azure Front Door + WAF	1. Mejor entrada global y baja latencia. 2. WAF centralizado protege contra ataques comunes.
Backend	App Service, Functions, AKS	AKS	1. Adecuado para microservicios con escalado independiente. 2. Soporta resiliencia, despliegues canary y control de runtime.
Arquitectura	Monolito modular, SOA, microservicios	Microservicios por dominio	1. Aísla dominios bancarios. 2. Permite escalabilidad y despliegue independiente.
Integración	HTTP síncrono total, ESB, eventos	APIs + eventos	1. HTTP para consultas inmediatas. 2. Eventos para auditoría, notificaciones y procesos no bloqueantes.
Mensajería	Storage Queues, Event Hubs, Service Bus	Service Bus	1. Requiere confiabilidad empresarial. 2. Soporta tópicos, colas, DLQ y procesamiento transaccional.
Eventos	Solo Service Bus, Kafka, Event Grid	Event Grid + Service Bus	1. Event Grid distribuye eventos de forma desacoplada. 2. Service Bus garantiza procesamiento confiable donde el mensaje es crítico.

Autenticación	Implicit, Password Grant, Auth Code + PKCE	Auth Code + PKCE	1. Evita exponer tokens en navegador. 2. Es práctica recomendada para SPA y móvil.
Identidad	Construir IdP propio, usar producto existente	Producto corporativo OIDC/OAuth2	1. Reduce riesgo de implementar seguridad crítica. 2. Centraliza gobierno de identidad, MFA y políticas.
Mobile	Flutter, React Native, nativo	Flutter	1. Consistencia multiplataforma. 2. Buen rendimiento y control de UI.
Cache	Sin cache, cache global, cache-aside	Redis + Cache-Aside	1. Reduce latencia y carga del Core. 2. Mantiene control sobre expiración e invalidación.
Lecturas frecuentes	Consultar Core siempre, CQRS, replicación total	CQRS + vistas materializadas	1. Optimiza consultas sin afectar comandos. 2. Mejora rendimiento para históricos y clientes frecuentes.
Auditoría	Logs normales, SQL común, ledger	Azure SQL Ledger / Confidential Ledger	1. Necesidad de inmutabilidad y no repudio. 2. Evidencia verificable ante auditorías.
Notificaciones	Un proveedor, varios canales directos, abstracción	Notification Service con Strategy	1. Permite fallback entre proveedores. 2. Desacopla canales del dominio transaccional.
Integración Core	Acoplamiento directo, Adapter, replicación	Adapter + Anti-Corruption Layer	1. Protege dominio digital de modelos legados. 2. Facilita cambiar proveedores o contratos.
Transferencias	Transacción distribuida 2PC, Saga, manual	Saga	1. Evita acoplamiento fuerte entre sistemas externos. 2. Permite compensaciones y trazabilidad.

Seguridad secretos	Variables de entorno, archivos, Key Vault	Azure Key Vault	1. Centraliza secretos y certificados. 2. Reduce exposición en código y pipelines.
DR	Backup manual, activo-pasivo, activo-activo	Activo-pasivo caliente	1. Balance adecuado entre costo y continuidad. 2. Permite evolución a activo-activo para consultas.
Observabilidad	Logs locales, APM externo, Azure Monitor	Azure Monitor + App Insights	1. Integración nativa con Azure. 2. Permite trazas, métricas, alertas y tableros operativos.

ANEXOS

DIAGRAMA C4 – CONTEXTO

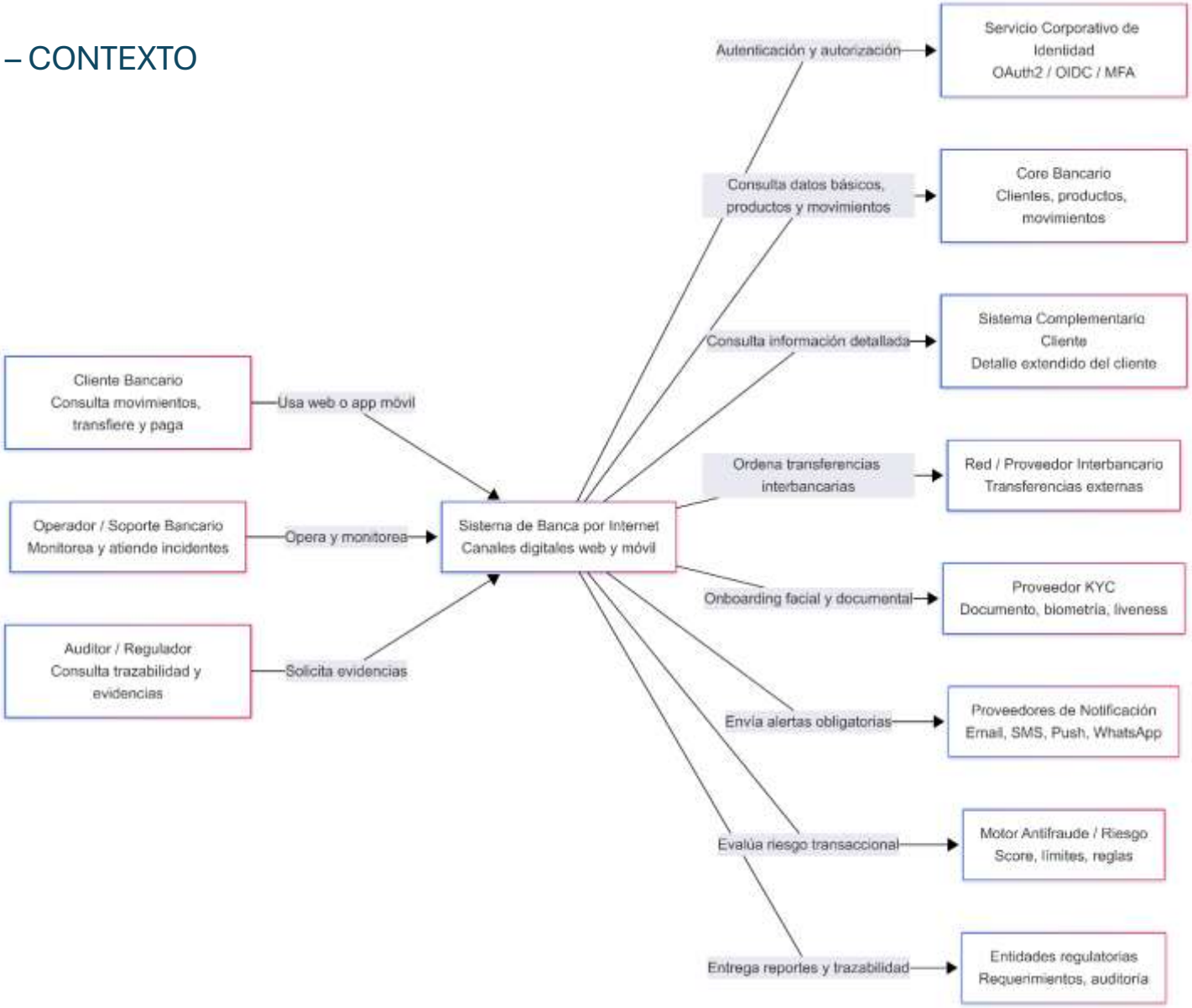


DIAGRAMA C4 – CONTENEDORES

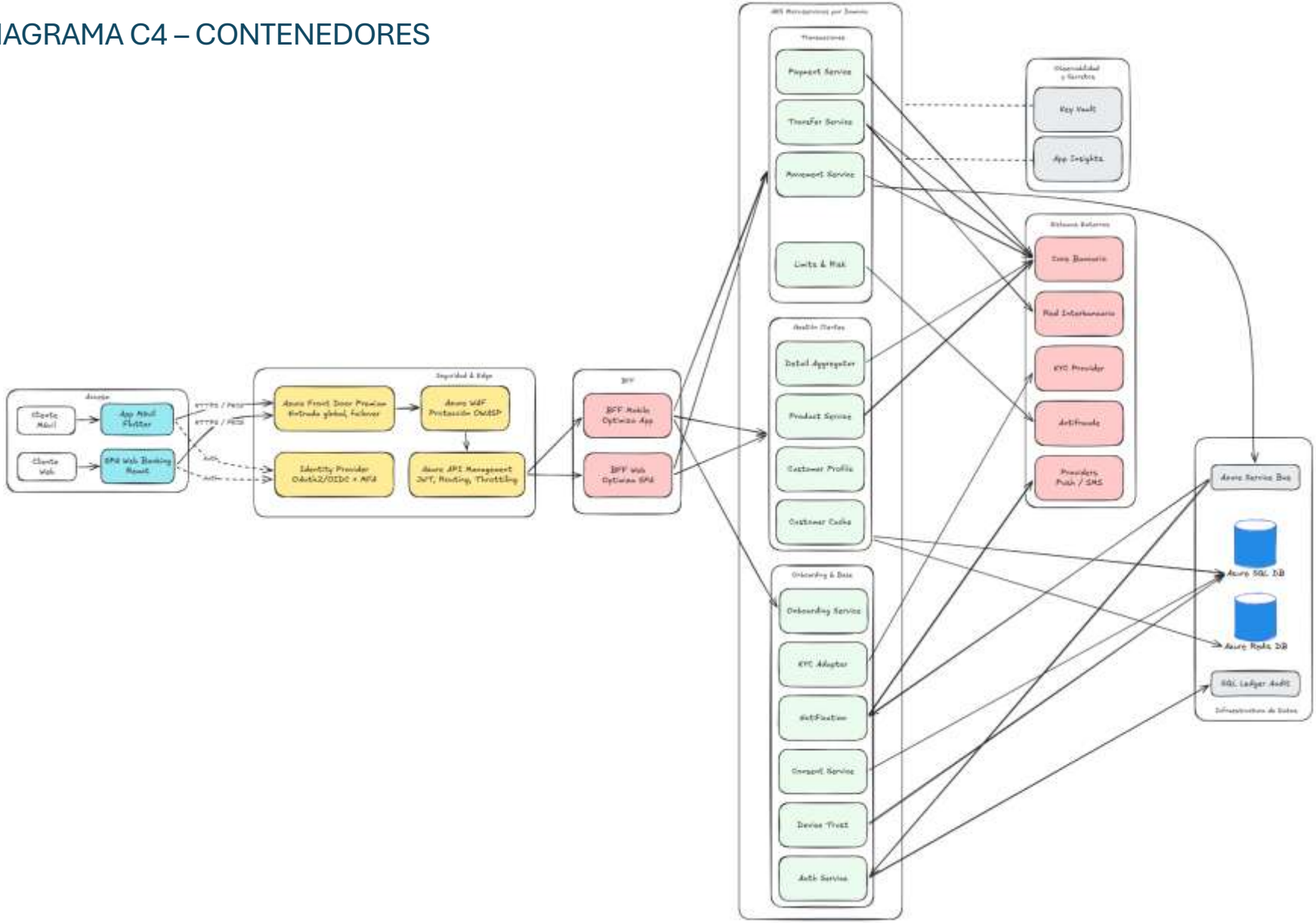


DIAGRAMA C4 – COMPONENTES

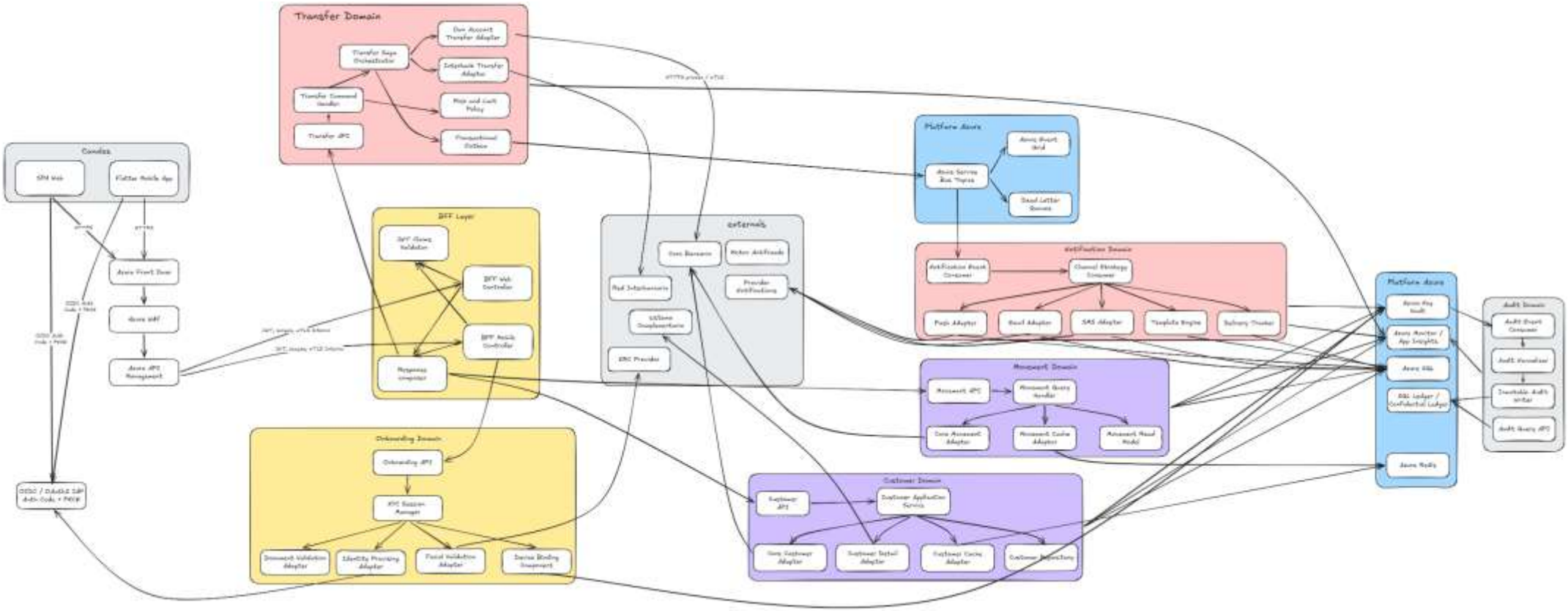
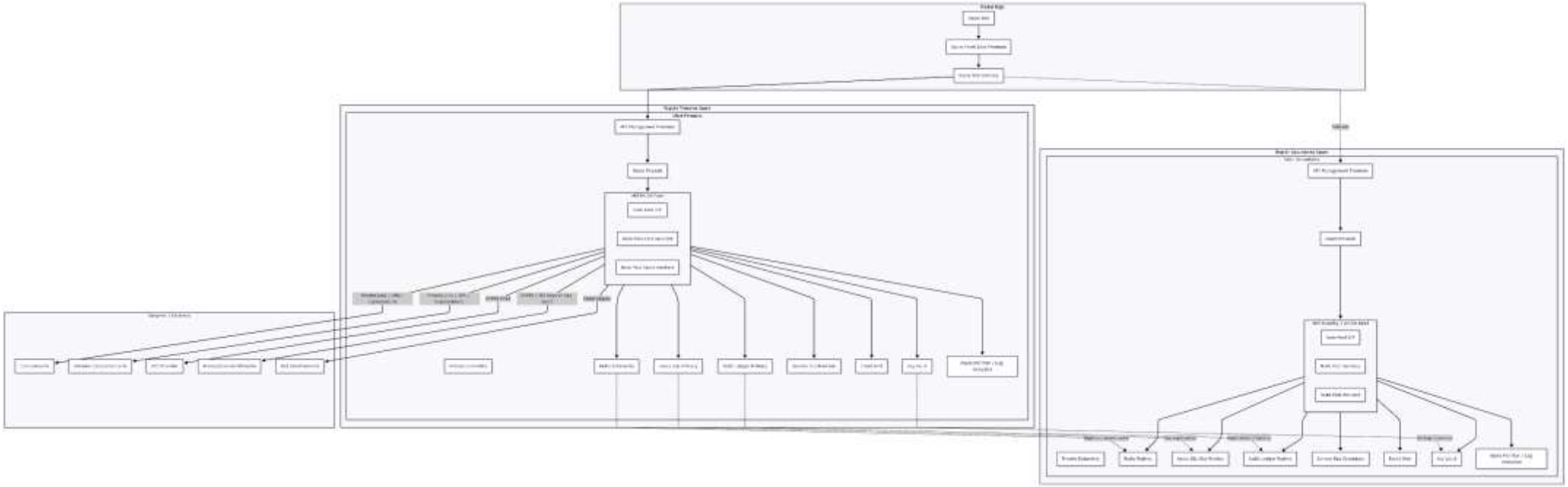
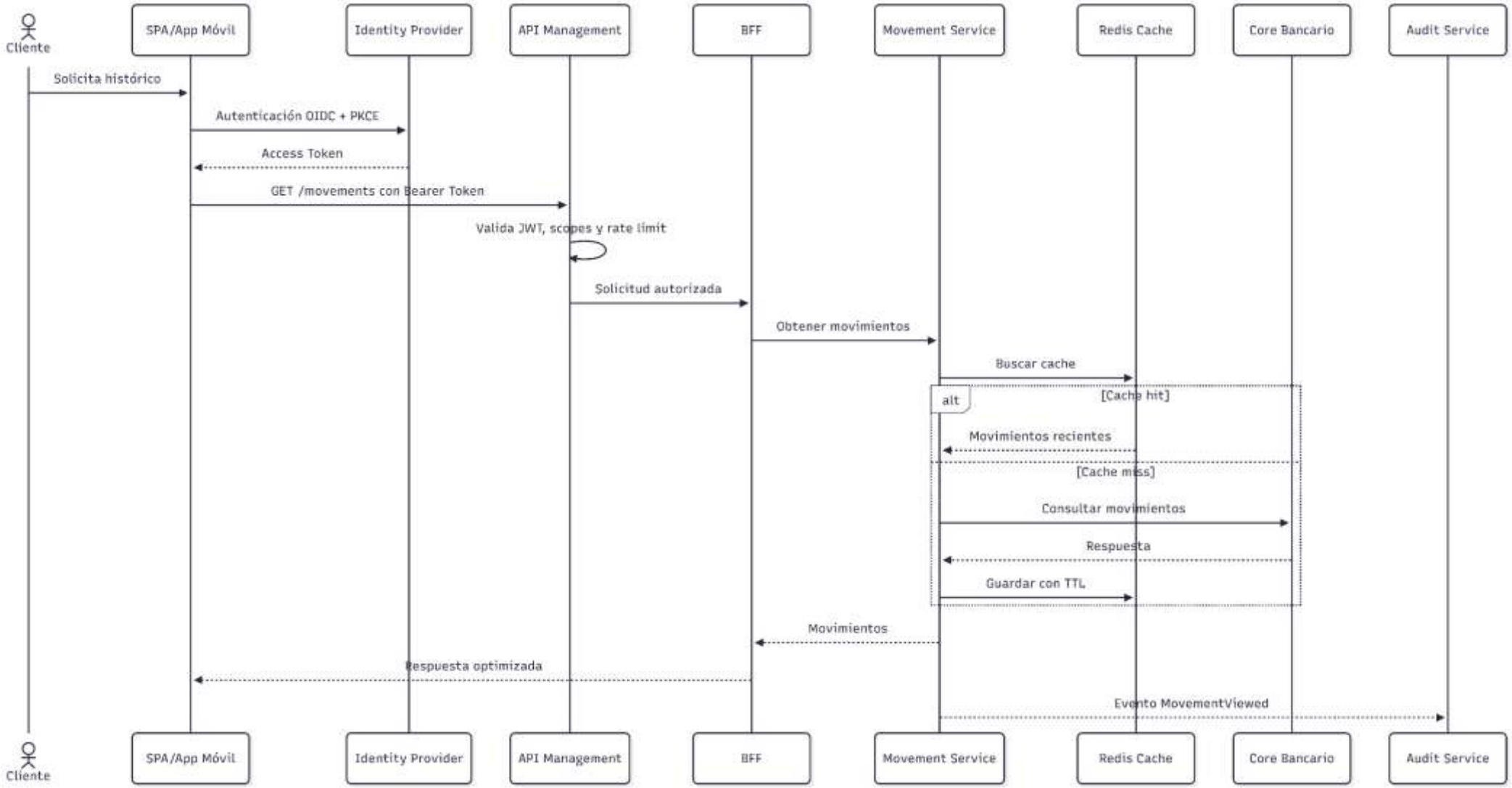
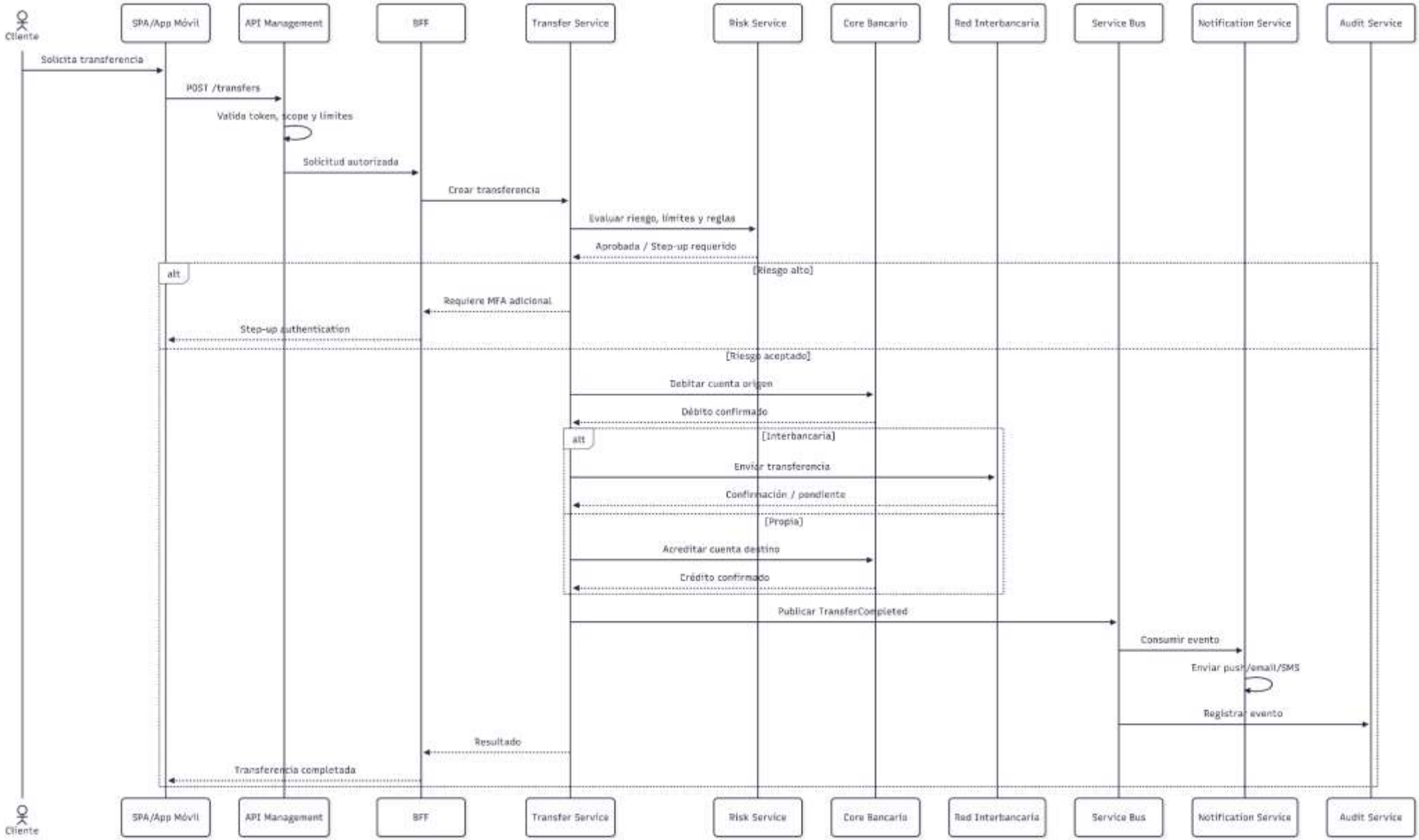


DIAGRAMA DE INFRAESTRUCTURA – AZURE



FLUJO DE CONSULTA DE MOVIMIENTOS



FLUJO DE TRANSFERENCIAS